1 Write a program that reads the contents of a text file and converts it into a coded version. For each word in the file, the program should reverse the order of its letters and then shift all vowels to the next vowel in a predefined sequence. The transformation must preserve the original formatting, including spaces, punctuation, and line breaks, and write the transformed text to a new output file.

Example

Original text (input file):

```
Hello world!
This is an example.
```



1. Reverse each word:

```
olleH dlrow!
sihT si na elpmaxe.
```

2. Shift vowels in each word (a → e, e → i, i → o, o → u, u → a):

- `olleH` → vowels: `o` → `u`, `e` → `i` → result: `ulliH`
- `dlrow` → vowels: `o` → `u` → result: `dlruw`
- `sihT` → vowels: `i` → `o` → result: `sohT`
- `si` → vowels: `i` → `o` → result: `so`
- `an` → vowels: `a` → `e` → result: `en`
- `elpmaxe` → vowels: `e` → `i`, `a` → `e`, `e` → `i` → result: `ilpmexi`

Transformed text (output file):

```
ulliH dlruw!
sohT so en ilpmexi.
```

2 Write a program that reverses the transformations applied by the first program, restoring the original text from the coded file. It should read the obfuscated file, reverse the vowel shifts, and reorder the letters in each word to their original sequence. The restored text must exactly match the original file, preserving all formatting, punctuation, and line breaks. The program should also be able to handle large files without loading the entire content into memory at once.

Example

Input file (obfuscated text from Q1):

```
ulliH dlruw!
sohT so en ilpmexi.
```

Step-by-step restoration:

1. **Reverse the vowel shift (e.g., e → a, i → e, o → i, u → o, a → u):**
   - `ulliH` → vowels: `u` → `o`, `i` → `e` → result: `olleH`
   - `dlruw` → vowels: `u` → `o` → result: `dlrow`
   - `sohT` → vowels: `o` → `i` → result: `sihT`
   - `so` → vowels: `o` → `i` → result: `si`
   - `en` → vowels: `e` → `a` → result: `an`
   - `ilpmexi` → vowels: `i` → `e`, `e` → `a`, `i` → `e` → result: `elpmaxe`
2. **Reverse each word back to its original order:**
   - `olleH` → `Hello`
   - `dlrow` → `world`
   - `sihT` → `This`
   - `si` → `is`
   - `an` → `an`
   - `elpmaxe` → `example`

Restored text (output file):

```
Hello world!
This is an example.
```

3 (*ArrayIndexOutOfBoundsException*) Write a program that meets the following requirements:

- Creates an array with `100` randomly chosen integers.
- Prompts the user to enter the index of the array, then displays the corresponding element value. If the specified index is out of bounds, display the message **Out of Bounds**.

4

Write a Java program that generates personalized certificates for multiple recipients using two external files. The first file is a text template containing placeholders in the form <<FieldName>>, such as <<Name>>, <<Course>>, <<Grade>>, and <<Date>>. The second file contains the corresponding values for each person. The program must read both files, replace all placeholders in the template with the actual values from the data file, and output each personalized certificate. Each generated certificate should be displayed on the screen and should have the option to be saved to a separate text file for each recipient. The program should allow running multiple times with different template files

CERTIFICATE OF COMPLETION

This is to certify that <<Name>>
has completed the course <<Course>>
with a grade of <<Grade>>.

Date of completion: <<Date>>

Congratulations!
Training Department